

CFD 2025-26 – Lab 07 – solution

The full code for this lab is provided in the notebook `CFDlab07.ipynb`.

For all the exercises, we define the following spaces and forms:

$$\begin{aligned} V_D &= \{\mathbf{v} \in [H^1(\Omega)]^2 \text{ s.t. } \mathbf{v} = \mathbf{u}_{\text{in}} \text{ on } \Gamma_{\text{in}}, \mathbf{u} = \mathbf{0} \text{ on } \Gamma_{\text{wall}}\}, \\ V_0 &= [H^1_{\partial\Omega \setminus \Gamma_{\text{out}}}]^2, \quad Q = L^2(\Omega), \\ a(\mathbf{u}, \mathbf{v}) &= \int_{\Omega} \nabla \mathbf{u} : \nabla \mathbf{v} \, dx, \quad b(\mathbf{v}, q) = - \int_{\Omega} \operatorname{div} \mathbf{v} \, q \, dx, \quad c(\mathbf{w}, \mathbf{u}, \mathbf{v}) = \int_{\Omega} (\mathbf{w} \cdot \nabla) \mathbf{u} \cdot \mathbf{v} \, dx, \end{aligned}$$

where $\mathbf{u}_{\text{in}}(x, y) = 4y(1 - y)\mathbf{i}$.

For the import of the mesh and the output of results, it is important to know the working directory in which your colab/jupyter notebook is running. The command `os.getcwd()` returns the path of current working directory: all relative paths used in input/output start from this point. Further instruction on changing the working directory are reported at the beginning of `CFDlab07.ipynb`. In this solution document, we assume that the current path contains two folders:

- `meshes`, containing the mesh files provided on WeBeep;
- `output`, where the output VTK files are saved (see below).

The different meshes are provided in the Gmsh¹ files `elbow1.msh` (miter joint), `elbow2.msh` (smooth turn), `elbow3.msh` (domain of Exercise 3), and they are loaded in Firedrake by the following command:

```
mesh = Mesh('meshes/elbow1.msh')
```

The labels of the different borders can be seen by plotting the mesh:

```
fig, ax = plt.subplots()
triplot(mesh, axes=ax)
ax.legend()
```

Exercise 1

Given $\mathbf{u}_{k-1}$, using the usual notation, the weak form of the fixed-point iteration problem is the following:

$$\begin{aligned} \text{find } \mathbf{u}_k &= \check{\mathbf{u}}_k + \mathcal{R}\mathbf{g} \in V_D, \text{ with } \check{\mathbf{u}}_k \in V_0, \text{ and } p_k \in Q \text{ such that} \\ a(\check{\mathbf{u}}_k, \mathbf{v}) &+ c(\mathbf{u}_{k-1}, \mathbf{u}_k, \mathbf{v}) + b(\mathbf{v}, p_k) - b(\mathbf{u}_k, q) \\ &= -a(\mathcal{R}\mathbf{g}, \mathbf{v}) - c(\mathbf{u}_{k-1}, \mathcal{R}\mathbf{g}, \mathbf{v}) + b(\mathcal{R}\mathbf{g}, q) \quad \forall (\mathbf{v}, q) \in V_0 \times Q. \end{aligned} \tag{1}$$

¹<https://gmsh.info>

The iterations are initialized with the solution of the Stokes problem on the same domain and with the same boundary conditions, whose weak form reads as follows:

$$\begin{aligned} \text{find } \mathbf{u}_k^{\text{Sto}} = \check{\mathbf{u}}_k^{\text{Sto}} + \mathcal{R}\mathbf{g} \in V_D, \text{ with } \check{\mathbf{u}}_k^{\text{Sto}} \in V_0, \text{ and } p_k^{\text{Sto}} \in Q \text{ such that} \\ a(\check{\mathbf{u}}_k^{\text{Sto}}, \mathbf{v}) + b(\mathbf{v}, p_k^{\text{Sto}}) - b(\mathbf{u}_k^{\text{Sto}}, q) = -a(\mathcal{R}\mathbf{g}, \mathbf{v}) + b(\mathcal{R}\mathbf{g}, q) \quad \forall (\mathbf{v}, q) \in V_0 \times Q. \end{aligned} \quad (2)$$

Starting from the provided code template, the function spaces and data are defined. For Dirichlet boundary conditions we use a strong imposition by means of `DirichletBC`, therefore we actually need to assemble just the left-hand side of (1)-(2). These forms are implemented in two functions: `initialization_problem_forms` for (2) and `nonlinear_iteration_forms` for (1), which take trial/test functions and data as input and return a bilinear form `a` (representing the left-hand side of the problems) and a functional `L` (representing the forcing term $\int_{\Omega} \mathbf{f} \cdot \mathbf{v} dx$). Indeed, as we have seen in previous labs, a right-hand side functional must always be provided to assemble a linear problem: in the solution of this Exercise, we will pass $\mathbf{f} = \mathbf{0}$ as a source term.

These functions are going to be used in the following

Algorithm 1 Fixed-point iteration algorithm

- 1: initialization: $k \leftarrow 0$, $err \leftarrow 0$, and $(\mathbf{u}_0, p_0) \leftarrow$ solution of Stokes problem (2);
 - 2: store initialized solution: $\mathbf{u}_{\text{old}} \leftarrow \mathbf{u}_0$;
 - 3: define the forms of problem (1) by `nonlinear_iteration_forms`, depending on $\mathbf{u}_{\text{old}}$
 - 4: **while** $err > tol$ and $k < k_{\text{max}}$ **do**
 - 5: $k \leftarrow k + 1$;
 - 6: solve linear problem (1) depending on $\mathbf{u}_{\text{old}} \rightsquigarrow \mathbf{u}_k, p_k$;
 - 7: plot/export solution $\mathbf{u}_k, p_k$;
 - 8: update stopping criterion: $err \leftarrow \frac{\|\mathbf{u}_k - \mathbf{u}_{k-1}\|_{H^1}}{\|\mathbf{u}_{k-1}\|_{H^1}} + \frac{\|p_k - p_{k-1}\|_{L^2}}{\|p_{k-1}\|_{L^2}}$;
 - 9: update old solution: $\mathbf{u}_{\text{old}} \leftarrow \mathbf{u}_k$ (with the `assign` method!);
 - 10: **end while**
-

You may notice that `nonlinear_iteration_forms` is called only once, at line 3 of the algorithm. Indeed, it is enough for the forms to be *constructed* as depending on $\mathbf{u}_{\text{old}}$, then the update of the variable $\mathbf{u}_{\text{old}}$ (line 9) automatically updates also the forms of problem (1).

To initialize the solution $\mathbf{u}_0, p_0$ (line 1), we call `initialization_problem_forms` and store its output (completed with boundary conditions) into a `LinearVariationalProblem` and a corresponding `LinearVariationalSolver` for the initial solution of the Stokes problem, which is then solved by

```
solver_init.solve()
```

Then (line 2), a `Function` `u_old` is created and *assigned* the values of $\mathbf{u}_0$ (see the code for a comment on assignment). Afterwards (line 3), we call `nonlinear_iteration_forms` and store its output into the `LinearVariationalProblem` `problem` and the `LinearVariationalSolver` `solver` (using the provided solver parameters). We do not call `solver.solve()`, yet: this will happen during the iterations within the while loop.

The fixed point iterations are finally run, according to Algorithm 1.

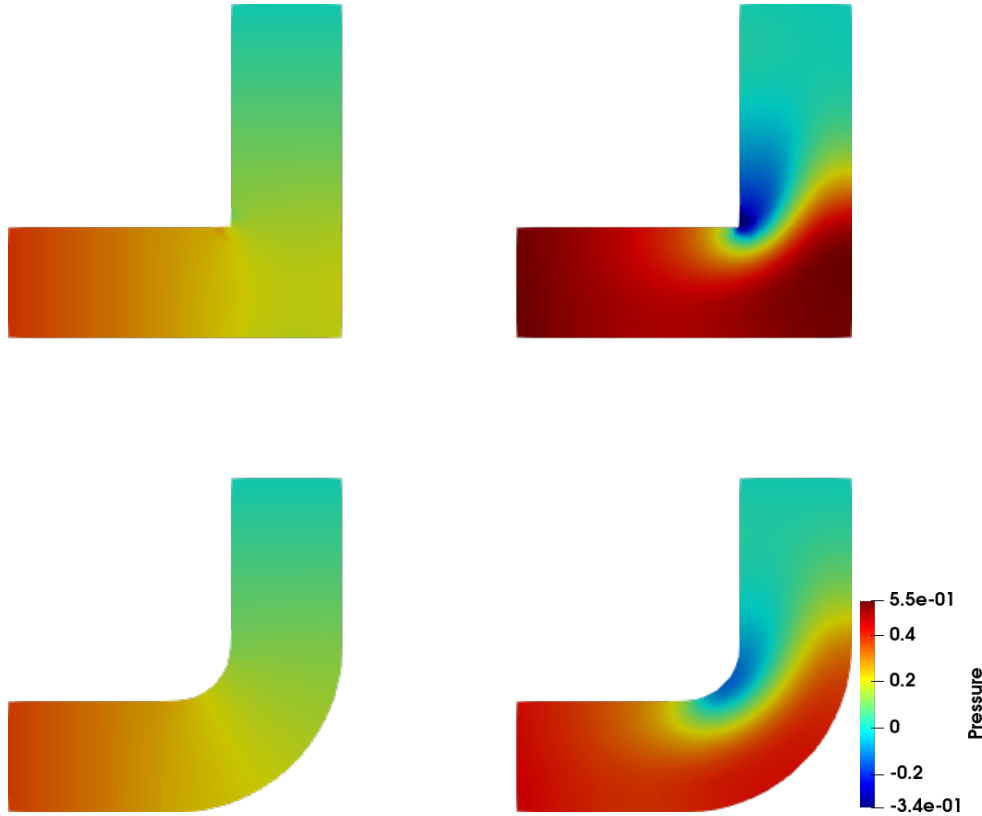


Figure 1: Exercises 1-2. Pressure of the Stokes (left) and Navier-Stokes (right) solution on the miter joint (above) and smooth turn (below).

Applying the solver to the solution of the problem over the miter joint, convergence of the fixed-point iteration algorithm is reached in 8 iterations, while for the smoother geometry the required iterations are 7. The resulting flow is displayed in Figures 1-2.

In the case of the miter joint `elbow1.msh`, we can notice two low-velocity recirculation regions at the external side of the turn and downwind of the concave corner. Near the concave corner, we can appreciate a sudden pressure drop, while at the external side of the turn pressure arises. This behavior is due to the inertia of the fluid, accounted for by the nonlinear advection term of the Navier-Stokes equations. Indeed, the corresponding Stokes solution does not show such features, and it is basically a Poiseuille flow except for a small neighborhood of the turning region.

Solving the problem in the smooth turn domain `elbow2.msh`, both the extension of the recirculation regions and the pressure drop are strongly reduced. Moreover, the overall pressure difference between the outlet and the inlet is reduced in the smooth domain (~ 0.4 instead of ~ 0.55), meaning that less energy (or better, work) is required to make the fluid flow through this pipe, with respect to the miter joint. A comparison with the corresponding Stokes solution can be done also in this geometry. The Stokes flow is essentially unperturbed by the smooth turn of the domain, with the parabolic profile that is maintained throughout the pipe and the pressure values that decrease linearly along the centerline.

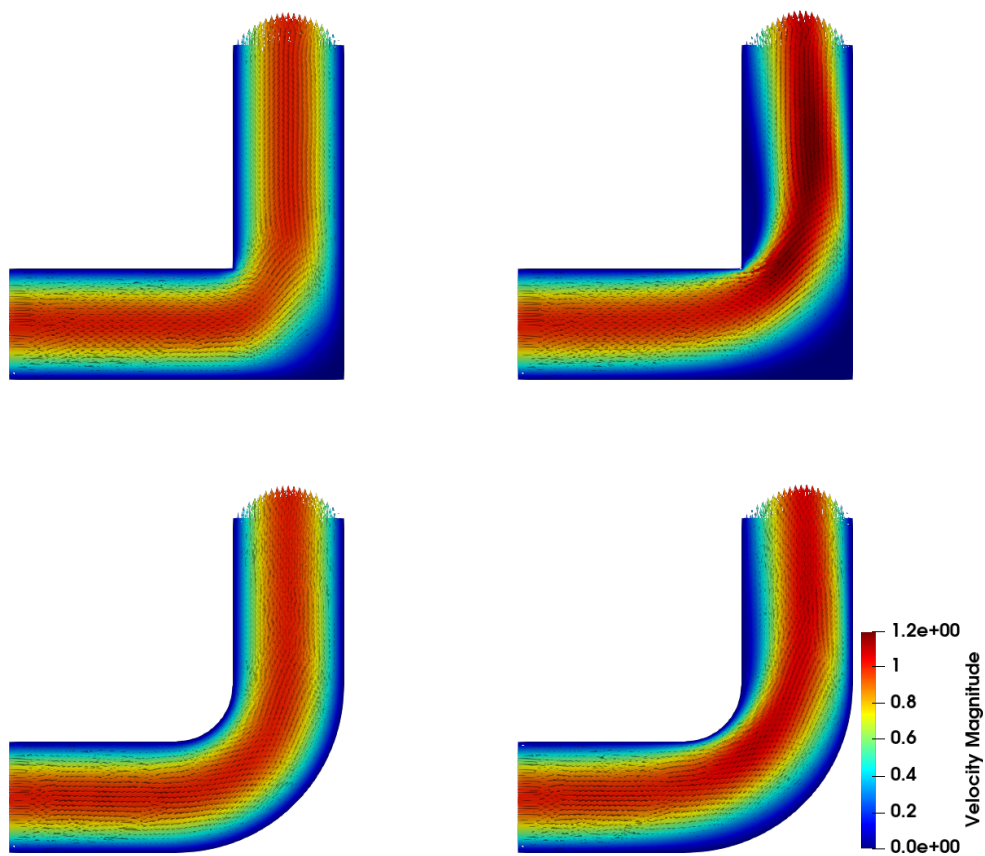


Figure 2: Exercises 1-2. Velocity of the Stokes (left) and Navier-Stokes (right) solution on the miter joint (above) and smooth turn (below).

Given $\mathbf{u}_{k-1}$, the weak form of the Newton iteration problem is the following:

$$\begin{aligned}
 &\text{find } \mathbf{u}_k = \check{\mathbf{u}}_k + \mathcal{R}\mathbf{g} \in V_D, \text{ with } \check{\mathbf{u}}_k \in V_0, \text{ and } p_k \in Q \text{ such that} \\
 &a(\check{\mathbf{u}}_k, \mathbf{v}) + c(\mathbf{u}_{k-1}, \check{\mathbf{u}}_k, \mathbf{v}) + c(\check{\mathbf{u}}_k, \mathbf{u}_{k-1}, \mathbf{v}) + b(\mathbf{v}, p_k) - b(\check{\mathbf{u}}_k, q) \\
 &\quad = c(\mathbf{u}_{k-1}, \mathbf{u}_{k-1}, \mathbf{v}) - a(\mathcal{R}\mathbf{g}, \mathbf{v}) - c(\mathbf{u}_{k-1}, \mathcal{R}\mathbf{g}, \mathbf{v}) - c(\mathcal{R}\mathbf{g}, \mathbf{u}_{k-1}, \mathbf{v}) + b(\mathcal{R}\mathbf{g}, q) \\
 &\quad \forall (\mathbf{v}, q) \in V_0 \times Q.
 \end{aligned} \tag{3}$$

To implement the Newton method, the only modification is the definition of the bilinear form and functional constructed in `nonlinear_iteration_forms`: all the rest of the code can stay the same.

The number of Newton iterations required to reach the prescribed tolerance on the relative increment is 4 for both geometries. The fact that this number is less than the number of fixed-point iterations is consistent with the well-known faster convergence of the Newton method. A far more significant difference may be found in more complex problems and this is the reason why Newton and quasi-Newton solvers are generally preferred in software libraries addressing actual industrial or biological problems.

The solution obtained at convergence is the same as in Exercise 1 (Figures 1-2).